

西安理工大学863数据结构 算法合集

1. 下列算法完成在顺序表SeqL的第i个位置插入元素x，正常插入返回1，否则返回0或-1，

```
int seq_ins(SeqList *SeqL,int i,int x) //seq_ins为顺序表插入函数
{
    //指向结构体的指针，DataType是数据类型
    int j;
    if (SeqL->len==maxsize)//判断顺序表的长度是否等于表长最大值，等于就满了
    {
        print("the list is full\n")           //输出表满了
        return 0;                             //表满，则返回0
    }
    else if(i<1||i>SeqL->len+1) //插入的位置为1~表的长度，超过这个范围
    {
        { print("the position is invalid\n"); //输出位置无效
          return -1;                         //位置无效返回-1
        }
        else
        {
            for(j=SeqL->len;j>=i;j--)//从后向前找i,j=i时表示找到i位置
                SeqL->data[j+1]=SeqL->data[j]//没找i位置，元素向后移动一位。
            SeqL->data[i]=x; //j=i时,i位置找到，将插入元素x放到SeqL->data[i]
            (SeqL->len)++;    //表的长度加1
            return 1;       //插入成功返回1
        }
    }
}
```

2. 下列算法完成删除顺序表SeqL的第i个元素，元素类型为DataType，其值通过参数px返回

```
int seq_del(SeqList *SeqL,int i, int *px )
{
    int j ;
    if (SeqL->len==0) //表空
    {
        printf("the list is empty\n");
        return 0;
    }
    else if(i<1||i>SeqL->len) //位置不对
    {
        {
            printf("\n the position is invalid");
            return -1;
        }
        else //正常删除
        {
            *px=SeqL->data[i]; //删除元素通过参数px 返回
            for (j=i+1;j<=SeqL->len;j++)
                SeqL->data[j-1]=SeqL->data[j]; //元素前移
            SeqL->len=SeqL->len-1; //表长减1
            return 1;
        }
    }
}
```

3. 设有一整型顺序表L，元素从位置1开始存放，下列算法实现将以第一个元素为基准，将其放置在表中合适的位置，使得其前面的元素都比它小，后面的元素均大于等于该元素。

```
void part(SeqList *L)
{
    int i,j; //循环变量声明*/
    int x;
    x=L->data[1]; //将第一个元素置入x中*/
    for(i=2;i<=L->len;i++)
        if( L->data[i]<x) //当前元素小于基准元素*/
```

```

61     {
62         L->data[0]=L->data[i];      /*当前元素暂存在0位置*/
63         for(j=i-1;j>=1;j--)        /*当前元素前面所有元素后移*/
64             L->data[j+1]=L->data[j];
65         L->data[1]=L->data[0]; /*当前元素从0位置移到最前面*/
66     }
67 }

```

4. 下列算法完成用头插法为数组a中的n个元素建立一个带头结点的单链表，并返回其头指针，请在空的下划线上填写合适的内容完成该算法。

```

70 NodeType *creat12(int a[],int n)
71 {
72     NodeType *head, *s;
73     int i;
74     head=(NodeType*)malloc(sizeof(NodeType)); /*为头结点申请空间*/
75     if(head->next=NULL); /*链表初始化为空*/
76     if(head==NULL) return NULL; /*初始化未成功，返回空*/
77     for (i=1; i<=n; i++)
78     {
79         s=(NodeType*)malloc(sizeof(NodeType));
80         s->data=a[n-i]; /*给新结点的数据域赋值*/
81         s->next=head->next; /*新结点的指针域指向头的下一个元素*/
82         head->next=s; /*头结点指向新结点*/
83     }
84     return head; /*返回头指针*/
85 }

```

5. 下列算法完成用尾接法为数组a中的n个元素建立一个带头结点的单链表，并返回其头指针

```

88 NodeType *creat11(int a[ ],int n)
89 {
90     NodeType *head,*tail,*s; /* head为头指针， tail为尾指针*/
91     int i;
92     head=init1( ); /*链表初始化*/
93     if (head==NULL) return NULL;
94     tail=head; /*尾指针初始化为头指针*/
95     for (i=0;i<n;i++)
96     {
97         s=(NodeType*)malloc(sizeof(NodeType)); /*为新结点申请空间*/
98         if (s==NULL) return NULL; /*申请不到空间，则返回空*/
99         s->data=a[i] ; /*给新结点的数据域赋值*/
100        s->next=NULL; /*给新结点的指针域赋值*/
101        tail->next=s; /*将新结点接到表尾*/
102        tail=s ; /*新结点为当前的表尾*/
103    }
104    return head; /*返回头指针*/
105 }

```

6. 已知整型带头结点的单链表H，下列算法实现将该链表中的元素倒置，若原链表中的元素为1, 2, 3, 4, 5; 倒置后在则变成5, 4, 3, 2, 1.

```

108 void reverse ( NodeType *H)
109 {
110     NodeType *p;
111     p=H->next; /*p指向首元结点*/
112     H->next=NULL; /*头结点的指针域为空*/
113     while( p!=NULL ) /*当p结点不为空时*/
114     {
115         q=p;
116         p=p->next; /*p指针后移*/
117         q->next= H->next; /*将q所指向的结点插入到头结点之后*/
118         H->next=q;
119     }
120 }

```

7. 设计一算法利用单链表中原来的结点将该单链表中元素倒置。即：原来的元素如果为：1，2，3，4，5；倒置后变成5，4，3，2，1。

```
void reverse (NodeType * H)
{
    NodeType *p;
    p=H-> next;
    H->next=NULL;
    while(p)
    {
        q=p; p=p->next;
        q->next=H->next;
        H->next=q;
    }
}
```

8. 设计一算法尽量利用原表中的结点，将该链表划分成两个链表，一个链表中放置正数和零， 另外一个链表中放置负数。

/*拆分单链表函数。正数和零仍放在head中,负数放入head1中，通过函数返回head1*/

```
NodeType *chaifen(NodeType *head)
```

```
{
    NodeType *p,*q,*r,*head1;
    head1=( NodeType *)malloc(sizeof(NodeType )); //为新表建立头结点
    head1->next=NULL;
    r=head1; //r为负数指针
    p=head; //p 为整数和0的指针
    while(p->next) // 从原表第一个结点开始
    {
        if(p->next->data<0)
        {
            q=p->next; //q指向负数结点
            p->next=q->next; //负数结点从原表取出
            r->next=q; // 负数结点以尾接法插入到新表
            r=q;
        }
        else
            p=p->next;
    }
    r->next=NULL; //新表尾结点的指针域为空
    return head1;
}
```

9. 设计一算法删除单链表中重复的结点。

```
void pur_LinkList(NodeType * H)
```

```
{
    NodeType *p,*q,*r;
    p=H->next;
    if(p==NULL)
        return;
    while(p->next)
    {
        q=p; //找与p重复的结点并删除
        while(q->next)
            if (q->next->data==p->data)
            {
                r=q->next;
                q->next=r->next; free(r);
            }
            else
                q=q->next;
        p=p->next;
    }
}
```

10. 设计一算法删除单链表中所有的元素x。

```
void pur_LinkList(NodeType* H)
```

```

181 {
182     NodeType *p,*q,*r;
183     q=H;
184     while(q->next!=NULL)
185     {
186         if (q->next->data==x)//找x并删除
187         {
188             p=q->next;
189             q->next=p->next;
190             free(p );
191         }
192         else q= q ->next;
193     }
194 }

```

11. 设计一算法尽量用原表中的结点将两个升序排列的单链表合并为一个升序排列的单链表。

```

197 LinkList merge(NodeType *A, NodeType * B)
198 {
199     NodeType *p,*q,*tail;
200     p=A->next;
201     q=B->next; //从两个表第一个结点开始
202     C=A;
203     C->next=NULL;tail=C;
204     free(B);
205     while(p&&q) //当两个表都有结点时
206     {
207         if(p->data<q->data) //取出两个表当前小的元素
208         {
209             s=p;
210             p=p->next;
211         }
212         else{s=q;q=q->next;}
213         tail->next=s; // 用尾接法插入到新表中
214         tail=s;
215     }
216     if(p==NULL)
217         p=q; //将剩余元素放入新表
218     tail->next=p;
219 }

```

12. 设计一算法尽量用原表中的结点将两个升序排列的单链表合并为一个降序排列的单链表。

```

222 LinkList merge(NodeType *A, NodeType *B)
223 {   NodeType* C,*p,*q;
224     p=A->next;
225     q=B->next; //从两个表第一个结点开始
226     C=A;
227     C->next=NULL;
228     free( B );
229     while(p&&q) //当两个表都有结点时
230     {
231         if(p->data<q->data) //取出两个表当前小的元素
232         {
233             s=p;
234             p=p->next;
235         }
236         else
237         {
238             s=q;
239             q=q->next;
240         }

```

```

241         s->next=C->next;    // 用头插法插入到新表中
242         C->next=s;
243     }
244     if(p==NULL)
245         p=q; //将剩余元素放入新表
246     while( p )
247     {
248         s=p;
249         p=p->next;
250         s->next=C->next;
251         C->next=s;
252     }
253 }
254 13. 设计一算法在带头结点的单链表L中，将新元素x插入到带头结点的单链表L中的
255 元素elm之后，若不存在元素elm，则插入到最后。
256 void link_ins(NodeType *head,int x,int elm)
257 {
258     NodeType *p,*s;
259     p=head;
260     while(p->next!=NULL&&p->next->data!=elm)
261         p=p->next;
262     s=( NodeType *)malloc(sizeof(NodeType ));
263     s->data=x;
264     s->next=NULL;
265     if(p->next==NULL) //没找到时，p指向最后一个结点
266         p->next=s;
267     else //找到时，p->next指向elm元素
268     {
269         s->next=p->next->next;
270         p->next->next=s;
271     }
272 }
273 14. 设计一算法在带头结点的单链表L中，将新元素x插入到带头结点的单链表L中的
274 元素elm之前，若不存在元素elm，则插入到最后。
275 void link_ins(NodeType *head,int x,int elm)
276 {
277     NodeType *p,*s;
278     p=head;
279     while(p->next!=NULL&&p->next->data!=elm)
280         p=p->next;
281     s=( NodeType *)malloc(sizeof(NodeType));
282     s->data=x;
283     s->next=NULL;
284     if(p->next==NULL) //没找到时，p指向最后一个结点
285         p->next=s;
286     else //找到时，p->next指向elm元素.p指向elm之前的结点
287     {
288         s->next=p->next;
289         p->next=s;
290     }
291 }
292 15. 设计一算法：将任意给定的一个元素x插入到升序排列的单链表中，使得该链表
293 中的数据依然有序。
294 void ins(NodeType* L, int x)
295 {
296     NodeType* p=L,*s,*q; //找第一个比x大的元素的前驱结点
297     while(p->next!=NULL&&p->next->data<x)
298         p=p->next;
299     /*若x比所有元素都大，p指向最后一个结点；否则指向第一个比x大的元素的前
300     驱结点，两种情况下，都是将x插入到p之后*/

```

```

301     s=(NodeType*)malloc(sizeof(NodeType)); //为新结点申请空间
302     s->data=x;
303     s->next=p->next; //新结点插到p之后
304     p->next=s;
305 }
306 16. 设计一算法，求该链表中值最大的结点并返回该结点的指针，若链表为空，
307 则返回一个空指针。
308 NodeType *maxNode(NodeType *L)
309 {
310     NodeType *p=L->next,*s;
311     if(p!=NULL)
312     {
313         s=p; //第一个结点初始化为最大的结点
314         p=p->next; //从第二个开始比较
315         while(p!=NULL)
316         {
317             if(p->data > s->data)
318                 s=p; //s指向当前最大的结点
319             p=p->next;
320         }
321         return s;
322     }
323     else
324         return NULL;
325 }
326 17. 设计一算法，统计二叉树中度为2的结点总数。
327 int n=0;
328 void count(BsTree *bt)
329 {
330     if(bt!=NULL)
331     {
332         if(bt->lchild!=NULL&&bt->rchild!=NULL)
333             n=n+1;
334         count(bt->lchild);
335         count(bt->rchild);
336     }
337 }
338 18. 设计一算法，查找元素x所在结点，若找到，返回该结点的指针，否则返回一个空
339 指针。
340 BiTree Search(Bstreec *bt, int x)
341 {
342     Bstree *p=NULL;
343     if (bt)
344     {
345         if(bt->data==x)
346             return bt;
347         if (bt->lchild!=NULL)
348         {
349             p=Search(bt->lchild,x);
350             if(p)
351                 return p;
352         }
353         if ( bt->rchild!= NULL)
354         {
355             p=Search(bt->rchild,x);
356             if(p)
357                 return p;
358         }
359     }
360     return NULL;

```

```

361 }
362 19. 设计一算法，统计叶子结点总数。
363 int countleafl(Bstree *bt)
364 {
365     if (bt==NULL)
366         return 0;
367     if(bt->lchild==NULL && bt->rchild==NULL)
368         return 1;
369     else
370         return(countleafl(bt->lchild)+countleafl(bt->rchild));
371 }
372 20. 设计一算法，求二叉树的深度。
373 int DeepBsTree(Bstree *T)                                //求二叉树的深度
374 {
375     int ldeep,rdeep;
376     if(!T)
377         return 0;                                        //空二叉子树深度为0
378     else
379     {
380         ldeep = DeepBsTree(T->lchild);
381         rdeep = DeepBsTree(T->rchild);
382     }
383     if(ldeep>rdeep)                                    //ldeep就是每个“二叉树”的左子树深度。
384         return ldeep + 1;                                //rdeep就是每个“二叉树”的右子树深度。
385     else
386         return rdeep + 1;
387 }
388 21. 给定一组整型值,将s插入到t为根的二叉排序树中
389 void insert(BsTree *t,Bstree *s)    //将s插入到t为根的二叉排序树中
390 {
391     if(s->data < t->data)
392     {
393         if (t->lchild==NULL)
394             t->lchild=s;
395         else
396             insert(t->lchild,s);
397     }
398     else
399     {
400         if (t->rchild==NULL)
401             t->rchild=s;
402         else
403             insert(t->rchild,s);
404     }
405 }
406 22. 给定一组整型值,-1 作为结束，二叉排序树的生成算法
407 Bstree *bstreecreate() /*二叉排序树的生成算法*/
408 {
409     int x;
410     Bstree *s,*t;
411     t=NULL;
412     printf("input the elements of bstree,end flag is -1\n");
413     scanf("%d",&x);
414     while(x!=-1)
415     {
416         s=(BsTNode*)malloc(sizeof(BsTNode));
417         s->data=x;
418         s->lchild=s->rchild=NULL;
419         if (t==NULL)
420             t=s;

```



```

421         else
422             insert(t,s);
423         scanf("%d",&x);
424     }
425     return (t);
426 }
427 23. 写出用直接插入排序进行升序排序的算法。（不用题库，高分比记P236）
428 void ins_sort (int R[ ],int n)
429 {
430     int i,j,temp;
431     for(i=1;i<n;++i)
432     {
433         temp=R[i];
434         j=i-1;
435         /*下面这个循环完成了从待排关键字之前的关键字开始扫描，如果大于
436         待排关键字，则后移一位*/
437         while(j>=0&&temp<R[j])
438         {
439             R[j+1]=R[j];
440             --j;
441         }
442         R[j+1]=temp; //找到插入位置，将temp中暂存的元素插入
443     }
444 }
445 24. 写出用冒泡排序进行升序排序的算法。（不用题库，高分比记P240）
446 void bubble_sort(int R[ ],int n)
447 {
448     int i,j,flag,temp;
449     for(i=n-1;i>=1;--i)
450     {
451         flag=0;
452         for(j=i;j>=1;--j)
453             if(R[j-1]>R[j])
454             {
455                 temp=R[j];
456                 R[j]=R[j-1];
457                 R[j-1]=temp;
458                 flag=1; /*如果没有发生交换，则flag的值为0;
459                 发生交换flag的值为1; */
460             }
461         if(flag==0)/*一趟排序过程中关键字位置没有发生变化，
462         ，则证明序列有序，排序结束*/
463             return;
464     }
465 }
466 25. 写出在顺序表中进行二分查找的非递归算法。
467 int Binary_Search(S_TBL *tbl,int x)
468 {
469     int low,high,mid;
470     low=1;
471     high=tbl->last;
472     while(low<=high)
473     {
474         mid=(low+high)/2;
475         if(x < tbl->data[mid].key)
476             high=mid-1;
477         else if(x > tbl->data[mid].key)
478             low=mid+1;
479         else
480             return mid;

```

```

481         }
482         return 0;
483     }
484 26. 写出在顺序表中进行二分查找的递归算法。
485 int BSearch(S_TBL *tbl, keytype x, int low, int high)
486 /*在下界为low, 上界为high的线性表tbl中折半查找关键字为kx的元素*/
487 {
488     int mid;
489     if(low>high)
490         return -1;
491     mid=(low+high)/2;
492     if(x==tbl->data[mid].key)
493         return mid;
494     if(x<tbl->data[mid].key)
495         return(BSearch(tbl,x,low,mid-1));
496     return(BSearch(tbl,x,mid+1,high));
497 }

```

```

498 27. 写出在二叉排序树中进行查找的算法。
499 BsTNode *SearchData(BsTree *t, int x)
500 /*在二叉排序树t上查找关键码为kx的元素,找到返回其所在及结点
501 的指针, 否则返回一个空指针 */
502 {
503     BsTNode *q=t;
504     while(q)
505     {
506         if(x>q->data.key)
507             q=q->rchild;          /* 在右子树中查找 */
508         else if(x < q->data.key)
509             q=q->lchild;         /* 在左子树中查找 */
510         else
511             return q ;          /* 查找成功 */
512     }
513     return NULL;                /* 查找不成功 */
514 }

```

28. 假设二叉排序树t的各个元素值为整型且均不相同, 请定义结点类型并设计一个算法按递减次序输出各元素的值。

```

517 typedef struct node
518 {
519     int data;
520     struct node *lchild,*rchild;
521 }Bstree;
522 void bianli(Bstree *bt)
523 {
524     if (bt!=NULL)
525     {
526         bianli(bt->rchild);
527         printf("%5d",bt->data);
528         bianli(bt->lchild);
529     }
530 }

```

高分笔记补充算法

29. 树的先序遍历

```

533 void preorder(BTNode *p)
534 {
535     if (p!=NULL)
536     {
537         Visit(p);    //假设访问函数Visit()已经定义过,其中包含了对结点p
538                      //的各种访问操作,如可以打印出p对应的数值
539         preorder(p->lchild); //先遍历左子树
540         preorder(p->rchild); //再遍历右子树

```

```

541
542     }
543 }
544 30. 中序遍历
545 void inorder (BTNode *P)
546 {
547     if(p!=NULL)
548     {
549         inorder(p->lchild);
550         Visit(p);
551         inorder(p->rchild);
552     }
553 }
554
555 31. 后续遍历
556 void postorder(BTNode *p)
557 {
558     if(p!=NULL)
559     {
560         postorder(p->lchild);
561         postorder(p->rchild);
562         Visit(p);
563     }
564 }
565 32. 简单选择排序
566 void SelectSort (int R[],int n)
567 {
568     int i,j,k;
569     int temp;
570     for(i=0;i<n;++i)
571     {
572         k=i;
573         /*这个循环是算法的关键，他从无序序列中挑出一个最小关键字*/
574         for(j=i+1;j<n;++j)
575             if(R[k]>R[j])
576                 k=j;
577         /*下面3句完成最小关键字与无序序列第一个关键字的交换*/
578         temp=R[i];
579         R[i]=R[k];
580         R[k]=temp;
581     }
582 }

```